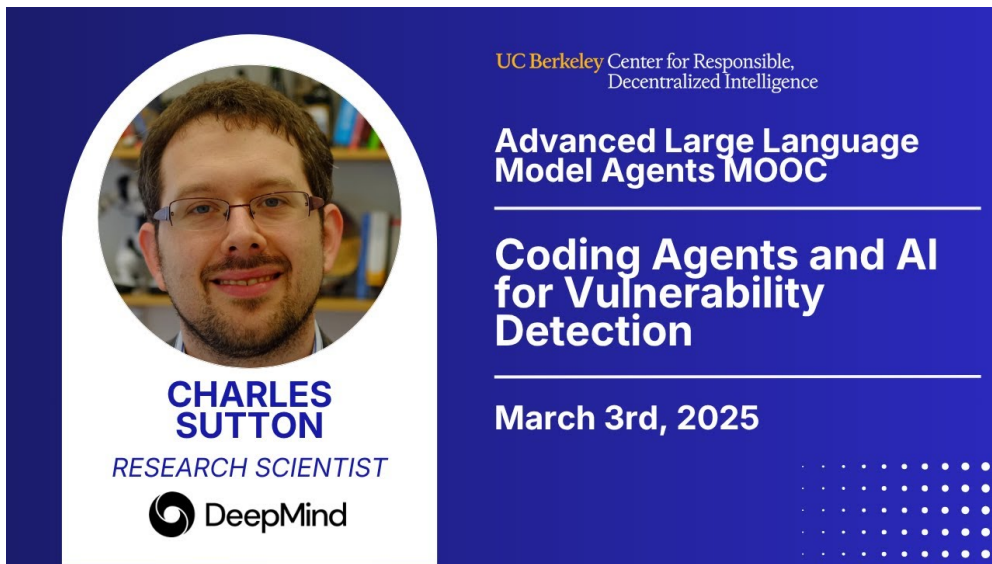


课程笔记

[LLM Agents SP25] Code Agents & AI Vulnerability Detection

基于 Charles Sutton 授课内容整理

2026-04-02



The graphic is a promotional slide for a MOOC. It has a dark blue background. On the left, there is a circular portrait of Charles Sutton, a man with glasses and a beard. Below the portrait, his name 'CHARLES SUTTON' is written in white, followed by 'RESEARCH SCIENTIST' in a smaller font, and the DeepMind logo. On the right, the text 'UC Berkeley Center for Responsible, Decentralized Intelligence' is at the top. Below it, 'Advanced Large Language Model Agents MOOC' is written in white. A horizontal line separates this from the main title 'Coding Agents and AI for Vulnerability Detection', which is also in white. Another horizontal line follows, and then the date 'March 3rd, 2025' is displayed. In the bottom right corner, there is a decorative pattern of white dots.

UC Berkeley Center for Responsible,
Decentralized Intelligence

Advanced Large Language
Model Agents MOOC

Coding Agents and AI
for Vulnerability
Detection

March 3rd, 2025

CHARLES
SUTTON
RESEARCH SCIENTIST
DeepMind

视频作者/频道: Berkeley RDI

发布日期: 2025-03-10

视频时长: 1:27:02

视频链接: <https://www.youtube.com/watch?v=JCK6qJtaCSU>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 课程定位与核心命题	3
1.1 本章小结	4
2 Agent 的本体：ReAct 循环、工具调用与测试时计算	4
2.1 从 buzzword 回到最小定义	4
2.2 测试时计算预算：继续修补还是重启轨迹	5
2.3 工具不是附属功能，而是能力边界	5
2.4 本章小结	6
3 评测体系演进：从 MBPP/HumanEval 到 SWE-Bench	6
3.1 早期代码评测的价值与失效	6
3.2 SWE-Bench 的突破	6
3.3 SWE-Bench 的局限同样重要	7
3.4 评测寿命与持续再造	8
3.5 本章小结	8
4 Harness Engineering：工具、接口与控制流共设计	8
4.1 为什么 harness 是一等公民	8
4.2 Agent Computer Interface (ACI) 视角	8
4.3 控制流设计谱系：从自由到约束	9
4.4 轨迹分析：必须看失败样本	10
4.5 本章小结	10
5 从代码修复到安全任务：能力迁移与场景重构	11
5.1 安全任务与普通修复任务的结构差异	11
5.2 CTF/Enigma 作为过渡台阶	11
5.3 动态分析工具进入主流程	11
5.4 本章小结	11
6 漏洞检测为什么难：语义上下文、评测噪声与动态验证	12
6.1 内核案例：局部可疑不代表全局成立	12
6.2 漏洞数据库驱动评测的天然难题	12
6.3 动态评测的必要性	13
6.4 从静态精度转向任务成功率	13
6.5 本章小结	13
7 Big Sleep 案例复盘：从假设到崩溃证据的完整闭环	13
7.1 项目目标与任务定义	13
7.2 三类核心工具	14
7.3 轨迹式问题求解	14
7.4 与传统 fuzzing 的比较	15
7.5 本章小结	16

8 工程落地蓝图：从课堂范式到可复现实验	16
8.1 先定义“可优化对象”，再优化模型	16
8.2 迭代节奏：按失败类型拆解系统瓶颈	16
8.3 控制流治理：探索效率与可审计性的平衡	17
8.4 安全落地的额外约束：责任披露与双重用途风险	17
8.5 研究机会：下一步值得投入的问题	18
8.6 本章小结	18
9 总结与延伸	19
9.1 整讲框架回顾	19
9.2 Further Reading	19
9.3 本章小结	19

1 课程定位与核心命题

本讲是 Berkeley “Advanced LLM Agents” 春季课程中一节高密度专题，由 Google DeepMind 的 Charles Sutton 主讲。讲座看似覆盖两个方向（Code Agent 与 AI Security），但真实主线只有一条：**如何把语言模型从“会写代码”推进到“能完成开放式软件任务”**。演讲结构按三段展开：先回到 coding agent 的评测与系统设计，再讨论 AI 在安全任务中的适配，最后用 Big Sleep 项目说明在真实漏洞挖掘里 Agent 化方法为什么成立。

全讲的工程判断标准

Sutton 的立场并不是“模型参数越大越好”，而是“系统能否闭环”。闭环包括：可执行任务定义、可验证结果、可迭代轨迹、可扩展工具接口。只要这四项里有一项缺失，Agent 结果就容易停留在演示层面。

演讲前半段多次强调一个容易被忽略的事实：Agent 研发不是单次 prompt 优化，而是在**巨大组合空间里做 hill climbing**。当团队不断改 system instruction、工具形态、反馈格式、后训练配比时，每一步都依赖评测分数作为导向。若评测目标错位，迭代方向就会整体偏航。

跨领域背景为什么重要

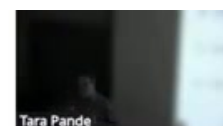
主讲人先讲自己的经历（NLP → 程序合成 → 代码生成 → 安全），不是铺垫个人简历，而是为了说明：AI 技术价值取决于你对问题域的理解深度。仅懂模型、不了解软件工程或安全流程，最终会把系统优化成“会答题”，而不是“会做事”。

不要把本讲理解成“更多安全 benchmark”

讲座的重点不是再介绍几个排行榜，而是解释为什么很多 benchmark 在早期有效、后期失效，以及为什么安全场景要求更强的动态验证。若只记住数据集名字，会错过最有价值的工程方法论。

Outline

- Coding Agents
- AI for computer security
- LLM agents for computer security



Presenting Slide

图 1: 讲座早段给出的三段式议程：Code Agent 基础、AI Security、Big Sleep 案例¹

1.1 本章小结

本章建立了读整节课的坐标系：这不是零散技术盘点，而是一堂关于“Agent 系统怎样被正确评估、正确约束、正确落地”的方法课。后续所有细节都服务于这个核心问题。

2 Agent 的本体：ReAct 循环、工具调用与测试时计算

2.1 从 buzzword 回到最小定义

Sutton 在 00:04 附近直接给出自己的定义：所谓 LLM Agent，本质是**动态计算 + 工具使用**。动态计算意味着模型不是一次输出最终答案，而是按任务难度分配更多测试时计算；工具使用意味着它可以把中间假设外化到可执行环境，通过命令、代码运行、调试信息不断修正下一步策略。

最小可运行 Agent 循环

$$\text{Trajectory}_t \xrightarrow{\text{LLM}} \text{Thought} + \text{Tool Call} \xrightarrow{\text{Environment}} \text{Observation}_{t+1} \xrightarrow{\text{append}} \text{Trajectory}_{t+1}$$

这个循环与 ReAct 思路一致：推理不是终点，而是为了决定下一次工具动作。

为什么“动态计算”比“单次聪明”更关键

- 复杂任务往往前几步就会犯错，允许迭代才能暴露并纠正错误路径。
- 同一模型在“多轮试错 + 可执行反馈”下，能力上限显著高于“一次生成”。
- 对真实工程问题，持续验证比单次文本流畅更有价值。

这套定义与课堂中反复出现的句子一致：“*Chain of Thought plus tool use is at the heart of agents.*”在 coding 场景里，工具输出就是下一轮推理的训练信号；在安全场景里，调试器、sanitizer、程序崩溃日志则成为更强的外部反馈。

¹ 视频画面时间区间：00:03:22–00:03:50。

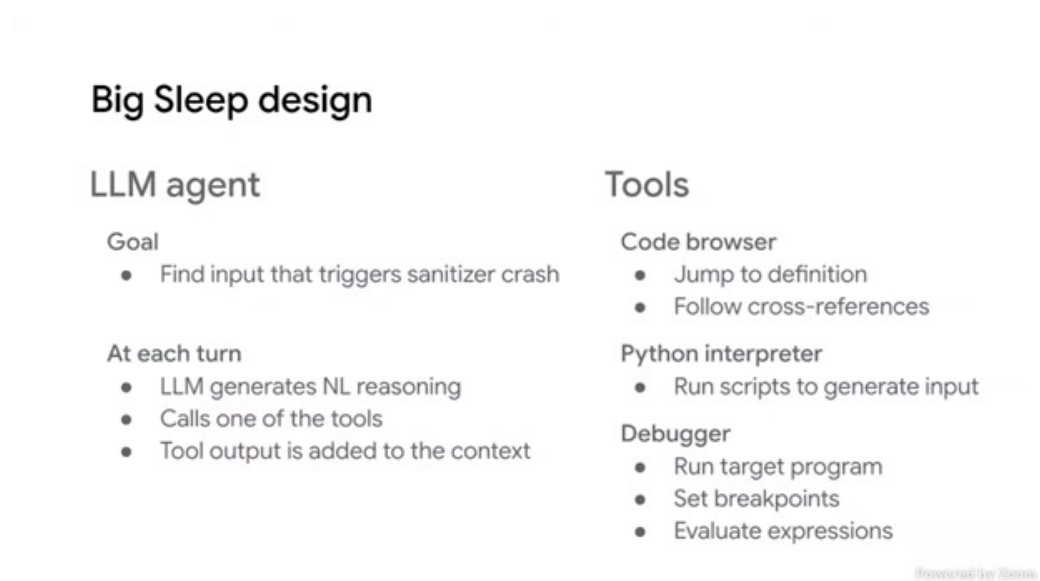


图 2: 课程后段再次回到 ReAct 风格循环: 观察、分析、调用工具、再观察²

2.2 测试时计算预算: 继续修补还是重启轨迹

讲座中一个很实用的问题是: 当得到一个“部分正确但未通过”的 patch, 应该继续在当前轨迹修补, 还是丢弃后重新采样? 这其实是在做**测试时计算预算分配**。对于短小改动, 重新采样可能更便宜; 对于复杂多文件变更, 保留上下文继续修补可能更优。

常见误区: 只看单条轨迹质量

如果只分析最好那条轨迹, 很容易高估系统。真实部署中应同时看:

- 每次成功平均消耗的轨迹数;
- 失败轨迹是否可恢复;
- 超时或循环失败的比例。

2.3 工具不是附属功能, 而是能力边界

在课堂表述里, 工具调用不是“给模型一点插件”, 而是直接决定问题可解空间。没有代码浏览工具, 模型几乎无法稳定定位大仓库问题; 没有执行反馈工具, 模型会长期停留在语义猜测; 没有调试能力, 安全场景中的漏洞触发链难以闭环。

工具调用的系统含义

一个 Agent 的能力上限并不只由 base model 决定, 还由“工具集合 × 反馈格式 × 控制策略”共同定义。主讲人把这三者称为需要一起调参的系统层面设计。

²视频画面时间区间: 01:18:06–01:18:40。

2.4 本章小结

Agent 在本讲中的定义很明确：它是一个可持续迭代的动作系统，而非长回答生成器。动态计算、工具反馈、可恢复轨迹构成了后续所有工程讨论的基础。

3 评测体系演进：从 MBPP/HumanEval 到 SWE-Bench

3.1 早期代码评测的价值与失效

MBPP、HumanEval 曾经推动了代码模型快速迭代，因为它们满足当时三项关键条件：难度合适、可自动判分、尚未广泛泄露。Sutton 直接指出，如今这类任务更像“code eval 的 MNIST”：仍可做基础对比，但难以代表真实软件工程能力。

Pass@K 的解释与部署含义

Pass@K 衡量“采样 K 次至少有一次通过测试”的概率。它对评估多样性有价值，但并不总等价于产品体验：

- IDE 单次补全更接近 Pass@1；
- Agent 带验证环节可以更接近 Pass@K；
- 因此指标是否有效，要看系统真实使用方式。

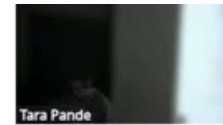
指标错配会造成方向性误导

如果产品场景只允许一次输出，却主要优化 Pass@K，团队可能会把系统训练成“样本池里有正确答案”，而不是“首答可用”。这种偏差在上线阶段会非常明显。

3.2 SWE-Bench 的突破

SWE-Bench 把任务从“写短函数”推进到“在真实代码库里修真实 issue”：输入是 issue 描述与仓库上下文，输出是 patch，验证依赖已有测试套件。相比早期评测，它显著提升了真实度，也迫使研究者正视代码导航、上下文检索、多文件编辑等系统问题。

Evaluation harnesses: SWE-Bench



Metadata

Repo	sympy / sympy	Issue #s	[17006]
Instance ID	sympy__sympy-17022	Pull Number	17022
Created At	Jun 9, 2019	Base Commit	b6fbc76

Problem Statement

Using lambdify on an expression containing an identity matrix gives us an unexpected result:

```
>>> import numpy as np
>>> n = symbols('n', integer=True)
>>> A = MatrixSymbol('A', n, n)
>>> a = np.array([[1, 2], [3, 4]])
>>> f = lambdify(A, A * Identity(n))
>>> f(a)
array([[1.+1.j, 2.+1.j],
       [3.+1.j, 4.+1.j]])
```

Instead, the output should be `array([[2, 2], [3, 5]])`, since we're adding an identity matrix to the array. Inspecting the globals and source code of `f` shows us why we get the result:

```
>>> import inspect
>>> print(inspect.getsource(f))
def _lambdifygenerated(A):
    return (1 + A)
>>> f.__globals__['I']
1
```

The code printer prints `I`, which is currently being interpreted as a Python built-in complex number. Printer should support identity matrices ...

Test Patch

```
sympy/printing/prettycode.py [...]
9 from sympy.logic import And, Or
10 from sympy.matrices import SparseMatrix, MatrixSymbol
11 from sympy.matrices import SparseMatrix, MatrixSymbol, Identity
12 from sympy.printing.prettycode import (
...
46 def test_numpyPrinter():
47     p = NumpyPrinter()
48     assert p._print(sign(x)) == 'numpy.sign(x)'
49     A = MatrixSymbol('A', 2, 2)
50     assert p._print(A**(-1)) == 'numpy.linalg.pinv(A)'
51     assert p._print(A**5) == 'numpy.linalg.matrix_power(A, 5)'
52     assert p._print(Identity(3)) == 'numpy.eye(3)'
```

Gold Patch

```
sympy/printing/prettycode.py
609 return "N/A" if (func, self._print(expr._tablist()))
610
611 = def _print_Identity(self, expr):
612     shape = expr.shape
613     if all([dim.is_Integer for dim in shape]):
614         return "N/A" if (self._module_format("numpy.eye"),
        self._print(expr.shape[0]))
615     else:
616         raise NotImplementedError("Symbolic matrix dimensions are not
        yet supported for identity matrices")
617 =
618 def _print_BlockMatrix(self, expr):
```

Powered by Sloom

图 3: 课程中对 SWE-Bench 任务形态与影响力的讲解画面³

SWE-Bench 真正改变了什么

它改变的不仅是分数，而是研发范式。团队开始系统性投入：

- 代码库级检索与导航；
- 执行反馈驱动修补；
- 多轮轨迹管理；
- Patch 生成与验证闭环。

3.3 SWE-Bench 的局限同样重要

Sutton 讲得很坦率：SWE-Bench 的价值与局限并存。数据来源集中在少量项目和特定语言生态；issue 描述风格与 IDE 内真实人机交互不完全一致；历史提交存在“一个 commit 混合多个改动”的噪声；测试集并非完美 verifier，可能出现“过拟合测试但语义错误”的补丁。

³视频画面时间区间：00:14:55-00:15:40。

维度	SWE-Bench 贡献	潜在偏差
任务真实性 验证方式	来自真实项目 issue/patch 链路 可自动运行测试，形成闭环评测	项目分布有限，领域覆盖不全 测试不能完整代表语义正确性
语言输入	保留真实 issue 文风与上下文 线索	与 IDE 内简短指令存在分布差异
训练价值	可驱动 retrieval、editing、execution 设计	历史提交可能缠绕多种改动目标

表 1: SWE-Bench 的“强现实性”与“强偏差性”并存

3.4 评测寿命与持续再造

讲座提出了一个非常关键的判断：**所有评测都有 shelf life**。起初难且未泄露，几年后常常变得易解且可被记忆。评测体系必须持续更新，否则系统会在过期目标上做高效优化。

一句可执行的方法论

“If your numbers aren’t good, you’re hill climbing on the wrong thing.” 翻译成工程动作就是：先检查评测目标是否真实、是否仍有区分度，再做模型或 harness 调参。

3.5 本章小结

从 MBPP/HumanEval 到 SWE-Bench，行业并不是“换了个榜单”，而是把代码智能从函数级推进到仓库级。与此同时，评测本身也必须被持续审计，否则高分不等于高可用。

4 Harness Engineering：工具、接口与控制流共设计

4.1 为什么 harness 是一等公民

Sutton 在 00:26 明确说出一句本讲高频引用的话：*“The design of the agent harness and the tools, you have to co-design it with whatever the model can do.”* 这句话意味着：我们不该把模型当黑盒再外挂工具，而应把工具接口、提示格式、控制流结构当成与模型同级的设计对象。

Harness Engineering 的三条工程原则

1. 动作语义要贴近模型熟悉分布，降低调用歧义；
2. 工具粒度要足够紧凑，减少“为一件事调用五个工具”；
3. 输出格式要可压缩且可追踪，避免上下文被噪声淹没。

4.2 Agent Computer Interface (ACI) 视角

课堂中引用了“ACI” (Agent Computer Interface) 概念，意在类比 HCI：人机界面设计有范式，智能体与执行环境的接口同样需要系统方法。比如“跳转到定义”、“按符号检索引用”这类动作，往

往比通用 grep 更符合模型思维路径，也更接近 IDE workflow。

工具设计的“熟悉路径”原则

主讲人借用了类似“Little Red Riding Hood principle”的说法：尽量把模型引导到后训练中见过的大量交互范式。工具名、参数、返回文本越贴近常见开发语境，越容易稳定调用成功。

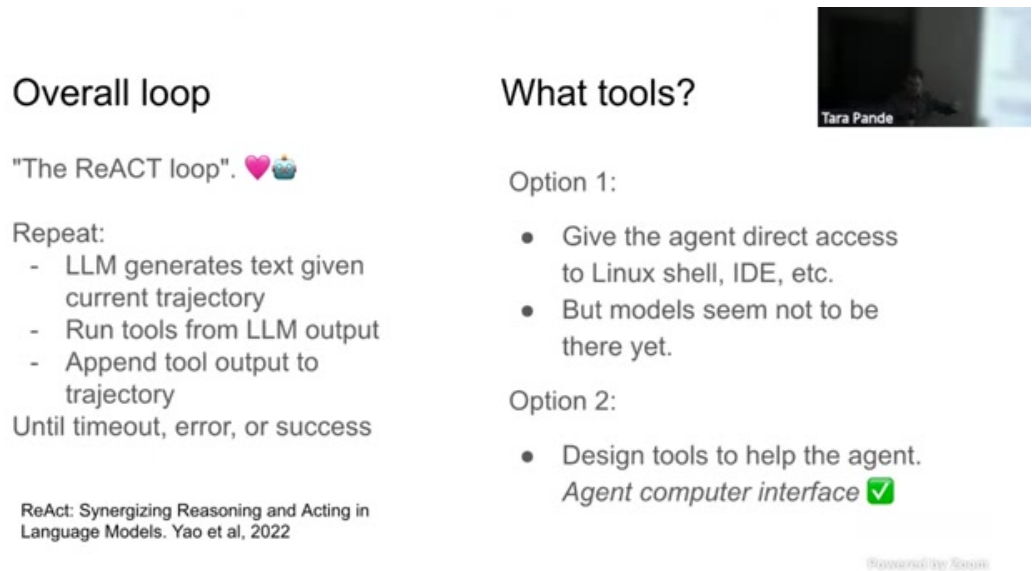


图 4: 课程中强调“harness 与工具必须联合设计”的关键画面⁴

4.3 控制流设计谱系：从自由到约束

讲座把 coding agent 控制流放在一条连续谱上：一端是“给 shell + IDE，几乎全动态”；中间是 SWE-agent 一类 ReAct 循环；再往右是分阶段拼接（先信息检索、后补丁生成）；最右是显式状态机（state machine）限制每一步可执行动作。

控制流范式	优点	风险	适配任务
全动态 ReAct	灵活探索、适应未知情况	轨迹漂移、成本不稳定	开放式定位与调试
两阶段拼接循环	信息收集与补丁生成解耦	阶段边界错了会级联失败	中等复杂修复任务
状态机约束循环	可控性强、便于审计与复现	搜索空间受限、可能错过解	高风险场景、合规流程

表 2: 控制流设计是任务风险与探索能力之间的权衡

⁴视频画面时间区间：00:26:24–00:26:55。

“控制越强越好”是误解

过强约束会让 Agent 失去在异常路径中的恢复能力；过弱约束又会产生长轨迹幻觉。正确做法是按任务复杂度与风险等级分层配置控制流，而不是追求单一范式。

4.4 轨迹分析：必须看失败样本

在安全 agent 讨论前，Sutton 重复了一个机器学习老原则：“Always look at the data.” 这里的“data”不是训练集，而是 Agent 执行轨迹。只有逐条查看失败路径，才能知道是检索错位、工具误用、还是验证器误导。

实操建议：轨迹驱动的 harness 迭代

- 每次版本迭代固定抽样失败轨迹做人工审阅；
- 将高频失败模式映射到新工具或新约束；
- 把“修完一次”的经验沉淀到 system instruction 与策略模板中。

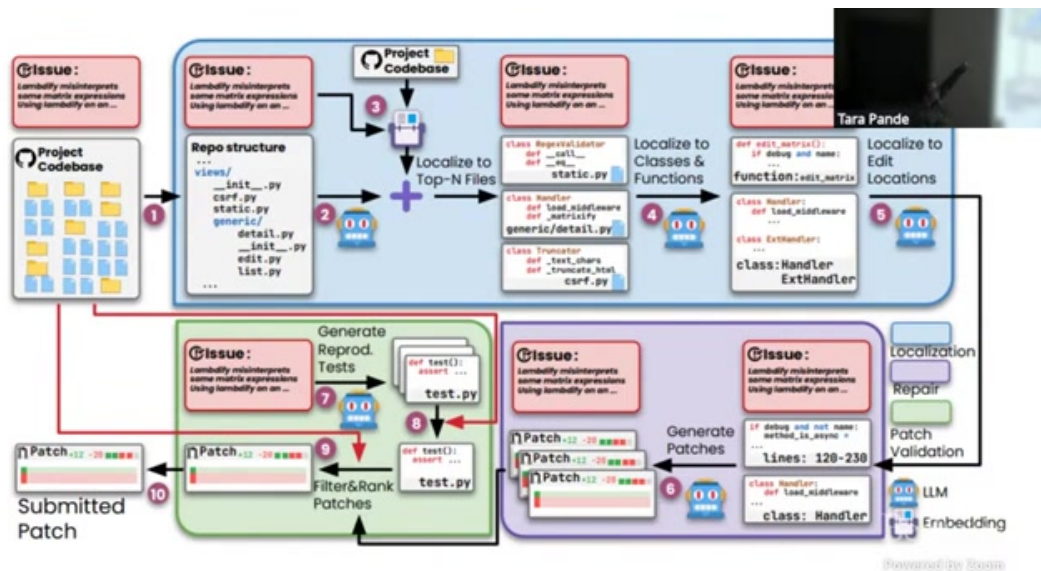


图 5: 围绕控制流与策略循环的讲解，强调动态循环和程序化约束并非对立⁵

4.5 本章小结

Harness Engineering 的核心是“系统协同”而非“Prompt 微调”。工具语义、反馈结构、控制流约束和模型能力必须联合优化，才可能在复杂任务中稳定提升。

⁵ 视频画面时间区间：00:38:10–00:38:55。

5 从代码修复到安全任务：能力迁移与场景重构

5.1 安全任务与普通修复任务的结构差异

安全任务不止“让测试通过”，还要求回答三个更难的问题：漏洞是否真实可触发、触发条件是什么、是否可跨路径复现。相比一般代码修复，漏洞挖掘更依赖动态行为观测、跨函数数据流推理和反事实验证。

安全 Agent 的最小任务闭环

1. 假设某路径可能存在越界/溢出/未检查条件；
2. 生成可执行输入或触发脚本；
3. 在 sanitizer/debugger 下运行并收集证据；
4. 失败则修正假设并继续探索，直到复现或证伪。

5.2 CTF/Enigma 作为过渡台阶

讲座给出 CTF 与 Enigma 类任务作为中间层：它们比 IDE 内日常补丁更强调利用链和触发链，但比真实大型系统漏洞仍可控。这个台阶有价值，因为它让团队能在可评测环境里迭代工具与控制流。

为什么先做 CTF 再做真实漏洞

CTF 允许在可重复环境里快速验证“工具是否有用”；真实漏洞则更多受上下文噪声、依赖环境、输入空间复杂度影响。先把 Agent 基础循环打磨好，再向真实系统迁移，成功率更高。

5.3 动态分析工具进入主流程

在 00:50 后半段，主讲人专门讲了 sanitizer 作为动态分析器的角色。与纯静态判断不同，sanitizer 在运行时直接提供“是否触发非法内存行为”的信号，这使它天然适合成为 Agent 的 oracle。

仅依赖静态分类器会陷入“看起来像漏洞”

很多函数局部看似危险，但全局调用链可能已做边界检查；反之，局部看似安全，跨模块组合后仍可能可利用。缺少动态验证时，误报与漏报都会显著增加。

5.4 本章小结

安全任务不是代码修复任务的简单放大，而是目标函数变化。只追求测试通过无法回答“是否真有可利用漏洞”，必须引入动态执行证据和更强的验证回路。

6 漏洞检测为什么难：语义上下文、评测噪声与动态验证

6.1 内核案例：局部可疑不代表全局成立

课程用内核代码示例解释了漏洞判定难点：某函数从包头读取长度后分配缓冲区并复制数据，若长度与真实 payload 不一致可能导致越界写。困难不在于模式识别本身，而在于需要确认可达路径、调用方约束和跨函数数据流。

```
5  /**
6   * tipc_crypto_key_rcv - Receive a session key
7   * @rx: the RX crypto
8   * @hdr: the TIPC v2 message incl. the receiving session key in its data
9   *
10  * This function retrieves the session key in the message from peer, then
11  * schedules a RX work to attach the key to the corresponding RX crypto.
12  *
13  * Return: "true" if the key has been scheduled for attaching, otherwise
14  * "false".
15  */
16 static bool tipc_crypto_key_rcv(struct tipc_crypto *rx, struct tipc_msg *hdr) {
17     struct tipc_crypto *tx = tipc_net(rx->net)->crypto_tx;
18     struct tipc_aead_key *skey = NULL;
19     u16 key_gen = msg_key_gen(hdr);
20     u32 size = msg_data_sz(hdr);
21     u8 *data = msg_data(hdr);
22     unsigned int keylen;
23
24     keylen = ntohs((__be32 *) (data + TIPC_AEAD_ALG_NAME));
25
26     spin_lock(&rx->lock);
27     if (unlikely(!rx->skey || (key_gen == rx->key_gen && rx->key.keys))) {
28         pr_err("%s: key existed <h>gen %d vs %d\\n", rx->name, rx->skey,
29               key_gen, rx->key_gen);
30         goto exit;
31     }
32
33     /* Allocate memory for the key */
34     skey = kmalloc(size, GFP_ATOMIC);
35     if (unlikely(!skey)) {
36         pr_err("%s: unable to allocate memory for skey\\n", rx->name);
37         goto exit;
38     }
39
40     /* Copy key from msg data */
41     skey->keylen = keylen;
42     memcpy(skey->alg_name, data, TIPC_AEAD_ALG_NAME);
43     memcpy(skey->key, data + TIPC_AEAD_ALG_NAME + sizeof(__be32), skey->keylen);
44
45     rx->skey_gen = key_gen;
46     rx->skey_mode = msg_key_mode(hdr);
47     rx->skey = skey;
48     rx->nokey = 0;
49     mb(); /* for nokey flag */
50
51     exit:
52     spin_unlock(&rx->lock);
53     /* Schedule the key attaching on this crypto */
54     if (likely(skey && queue_delayed_work(tx->wq, &rx->work, 0))) return true;
55     return false;
56 }
57
```

- Cannot understand without global information
 - e.g., maybe this is already checked

Portion by Atom

图 6：内核漏洞讲解片段：同一函数的风险取决于全局调用上下文⁶

主讲人的关键判断

“This function on its own is not vulnerable or non-vulnerable.” 单函数分类往往不是良定义问题。是否漏洞成立，取决于调用方是否已约束输入、路径是否可达、运行环境是否满足触发条件。

6.2 漏洞数据库驱动评测的天然难题

公开漏洞库（如 CVE 生态）看起来是理想数据源：有编号、有描述、有补丁。但将其直接转换为“函数级分类任务”会遇到严重问题：输入上下文该截到哪里、正负样本如何定义、补丁是否只对应一个根因、标签是否跨版本稳定。

“下载漏洞库就能训练检测器”通常过于乐观

课堂明确指出，这是最自然但也最容易踩坑的路径。若不先定义严谨任务边界，模型可能学到项目风格、提交习惯或模板语句，而不是真正的可利用性语义。

⁶视频画面时间区间：01:06:30–01:07:10。

6.3 动态评测的必要性

Sutton 提到 DARPA 相关竞赛与 Meta 安全评测方向，核心共同点是从静态标签转向可执行验证。对于漏洞发现任务，动态评测更接近真实目标：输入触发、运行证据、崩溃位置、可复现性。

Fuzzing 与 Agent 的互补关系

- Fuzzing 擅长高吞吐随机探索；
- Agent 擅长语义引导与路径假设；
- 二者结合可形成“广覆盖 + 深路径”的混合搜索。

这也是讲座后段反复强调的方向：AI 不替代 fuzzing，而是把搜索推向 fuzzer 难以触达的深层逻辑分支。

6.4 从静态精度转向任务成功率

安全场景里，最终指标不该是“这段代码像不像漏洞”，而应是“是否能构造触发输入并给出可审计证据”。这会把研究焦点从文本分类转到**可执行任务成功率**，与 Agent 范式天然一致。

对研究设计的直接启示

未来安全 Agent 论文若只报告静态分类分数，很难支撑工程价值；至少应增加动态触发率、复现稳定性、误报处置成本等指标。

6.5 本章小结

漏洞检测难在“上下文依赖 + 可执行验证”，而不是“模式匹配不足”。这也解释了为什么安全方向更需要 Agent 化系统，而不只是更强的代码文本模型。

7 Big Sleep 案例复盘：从假设到崩溃证据的完整闭环

7.1 项目目标与任务定义

Big Sleep（课程中也关联 Project Naptime）聚焦一个可明确验证的目标：给定代码库，生成输入触发 sanitizer crash。这个目标把“发现漏洞”转化为可执行搜索问题，既能自动评估，也能保留工程真实性。

Big Sleep: Agents for Vulnerability Detection

Find vulnerabilities with language models that can think and act like a security researcher.

- Navigate code
- Hypothesize vulnerabilities
- Test if correct by running code

Dynamic analysis vs static

Miltos Allamanis, Martin Arjovsky,
Charles Blundell, Lars Buesing, Mark
Brand, Sergei Glazunov, Dominik Maier,
Petros Maniatis, Guilherme Marinho,
Henryk Michalewski, Koushik Sen,
Charles Sutton, Vaibhav Tulsyan, Marco
Vanotti, Theophane Weber, Dan Zheng

<https://googleprojectzero.blogspot.com/2024/06/project-naptime.html>
<https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html>

Powered by Zoom

图 7: Big Sleep 项目介绍画面：将漏洞发现转为可执行触发任务⁷

Big Sleep 的关键建模

目标不是让模型口头解释“这里有漏洞”，而是让系统产生一个可复现输入，使程序在 sanitizer/调试器下产生客观异常。换言之，结论必须被执行环境认可。

7.2 三类核心工具

课程展示了该系统的三类工具：

- 代码浏览工具：按符号检索、跳转定义、跟踪引用；
- 输入生成工具：让 Agent 写 Python 程序构造复杂输入，而非手写长字节串；
- 调试执行工具：运行目标程序并返回 watchpoint/breakpoint 对应表达式值。

为什么让 Agent 生成 Python 输入程序

在安全场景，输入常包含二进制结构、嵌套格式或校验字段。让 Agent 写“生成器程序”比直接输出原始字节更稳健，也更利于后续迭代修改。

把调试器当交互式聊天窗口并不高效

课堂中的工具设计并非每步停住人工式单步调试，而是“整程序运行 + 回传命中的观察点值”。这减少了交互开销，同时保留关键证据，更适合自动化循环。

7.3 轨迹式问题求解

课程演示的轨迹非常典型：先做代码浏览并形成漏洞假设，然后输出漏洞报告与利用思路；接着生成输入并运行；若未崩溃则归因失败原因、修正输入、再次运行，直到触发异常。这个过程体

⁷ 视频画面时间区间：01:14:58–01:15:35。

现了 Agent 的核心优势：失败不是终点，而是下一轮搜索的监督信号。

Summary (security part)

- In many ways, wide open area
- Agentic techniques seem particularly natural
- Can require larger-scale understanding of software or system
- Many areas are only starting to be explored
 - e.g., network security, red teaming
- Moving from CtFs to real tasks

Powered by Slides

图 8: 讲座收尾对安全 Agent 现状与前景的总结⁸

结果层面的关键事实

课程披露了一个真实世界结果：在 SQLite 中发现并披露了真实漏洞。更重要的是，项目还展示了 variant analysis 能力，即基于已知漏洞模式在同项目内寻找相似变体。

7.4 与传统 fuzzing 的比较

主讲人提到一个很有说服力的细节：在知道漏洞位置后，再用通用 fuzzer 反向尝试，长时间运行（课程中提到约 150 小时量级）仍很难直接找到同一漏洞。这个对比说明 Agent 语义引导在深层路径探索中可能提供增益。

Agentic 搜索的潜在价值

- 可利用先验知识（历史漏洞、补丁语义）做定向搜索；
- 可在失败后重写输入生成策略；
- 可结合调试反馈缩小可疑路径空间。

不要把单案例成功外推为“全面超越”

Big Sleep 证明了可行性，但不意味着所有项目、所有漏洞类型都已可自动化。讲座结论是“刚起步、空间很大”，而不是“问题已解决”。

⁸视频画面时间区间：01:25:45–01:26:20。

7.5 本章小结

Big Sleep 的价值在于示范了完整闭环：目标可执行、工具可调用、轨迹可迭代、结果可验证。它把安全 Agent 从概念验证推进到可复现工程实践。

8 工程落地蓝图：从课堂范式到可复现实验

8.1 先定义“可优化对象”，再优化模型

这节课最值得迁移到日常研发的一点是：**先把任务定义成可执行、可验证、可迭代的对象，再谈模型优化**。很多团队失败并不是模型不够强，而是任务定义不完整，例如只有自然语言目标、没有明确 verifier，或只有最终分数、没有轨迹级诊断。

安全 Code Agent 的最小实验单元

一个可复现实验至少应包含六个组件：

1. 任务规范：输入、输出、约束、超时；
2. 执行环境：可重建容器与依赖版本；
3. 工具接口：代码浏览、运行、调试、输入生成；
4. 验证器：测试集或 sanitizer/oracle；
5. 轨迹日志：每轮 thought/tool/observation；
6. 评测脚本：批量运行与统计汇总。

若缺失第 5 项和第 6 项，系统往往只能“看到结果、看不到过程”，导致问题复现困难。Sutton 讲“always look at the data”的真正落点就是这里：团队要能把失败轨迹当作一等数据资产持续分析。

从课堂到工程的桥接策略

可以把课程中的三段结构映射成三层研发流程：

- 基座层：代码任务评测（MBPP/HumanEval/SWE-Bench）用于建立基础能力；
- 系统层：harness、工具、控制流用于提升复杂任务稳定性；
- 应用层：安全任务中的动态验证与证据链用于检验真实价值。

8.2 迭代节奏：按失败类型拆解系统瓶颈

课堂里多次提到“组合优化”，工程上可以进一步落地为失败分类。只要把失败原因稳定归类，调参优先级就会清晰得多，而不是盲目替换模型或堆叠 prompt。

失败类型	典型症状	优先修复方向
检索失败	反复浏览无关文件，定位路径漂移	提升符号级检索、加入跨引用导航
执行失败	命令调用格式错、环境依赖缺失	工具参数约束、容器基线固化
验证失败	测试通过但语义错误，或误判崩溃原因	强化 verifier、增加反例回放
策略失败	长轨迹自我强化错误假设	控制流重置、阶段化循环、状态机约束
成本失败	成功率提升但 token/时延不可接受	预算管理、早停策略、分层模型路由

表 3: 将 Agent 失败模式映射到可操作的系统改进方向

只做“成功案例复盘”会误导团队

如果评审会议只展示 best trajectory，系统会被优化成“偶尔很惊艳”。要提升线上可靠性，必须固定审查长尾失败样本，尤其是“看起来快成功但最终崩坏”的轨迹。

8.3 控制流治理：探索效率与可审计性的平衡

从课程给出的连续谱来看，控制流不是选边站，而是分场景配置。探索阶段可偏动态，收敛阶段可偏约束。对安全任务而言，还需额外满足审计要求：每个关键动作可追踪、每次结论可回放。

建议的双模式控制流

- 探索模式：高采样、多路径、宽工具权限，用于发现潜在漏洞路径；
- 验证模式：低采样、强约束、固定观察点，用于复现和证据固化。

两种模式共享同一任务目标，但资源策略和可解释性要求不同。

可执行的预算管理规则

可以把测试时预算拆成三层：

- 单轨预算：限制每条轨迹最大步数与最大运行次数；
- 任务预算：限制每个样例的轨迹总数与总时长；
- 批次预算：限制整轮评测的总 GPU/CPU 小时。

当某一层触发阈值时，系统必须输出“为什么停止”的结构化原因，避免黑箱超时。

8.4 安全落地的额外约束：责任披露与双重用途风险

Big Sleep 案例说明“发现漏洞”已经具备现实可行性，因此落地时不能只谈技术指标。安全 Agent 需要内建责任边界，包括漏洞处理流程、对外披露节奏、以及防止能力被直接转用于攻击自

动化。

双重用途风险必须前置治理

越是自动化的漏洞发现系统，越要把访问权限、执行沙箱、结果脱敏、披露审批做成默认流程，而不是项目后期补丁。否则技术成功会直接转化为合规风险。

实践中的最小安全治理清单

1. 所有目标仓库与运行环境白名单化；

2. 默认只在隔离执行环境运行 exploit-like 输入；

3. 发现疑似漏洞后先内部验证再进入披露流程；

4. 输出报告区分“证据”与“推断”，避免夸大结论；

5. 对外发布时去除可直接复现攻击链的敏感细节。

8.5 研究机会：下一步值得投入的问题

Sutton 结尾强调这是 “wide open area”，如果转成研究议程，可以落在四类问题：更强动态 verifier、更稳定跨仓库泛化、更低成本长轨迹搜索、更可信的自动证据生成。它们共同决定安全 Agent 能否从单点成功走向规模化稳定产出。

问题域	当前瓶颈	潜在突破方向
动态验证器	触发信号不稳定、误报判定成本高	组合 sanitizer + 语义断言 + 差分执行
跨项目泛化	工具和提示高度依赖单仓库习惯	可迁移 ACI 协议与任务归一化接口
搜索效率	长轨迹成本高且失败回报稀疏	分层规划、经验回放、失败记忆复用
证据自动化	报告可读性与可审计性不足	结构化漏洞证据模板与可回放 artifact

表 4: 安全 Agent 的中短期研究机会图谱

8.6 本章小结

把课堂方法迁移到工程实践的关键是两件事：第一，任何优化都要围绕可执行闭环；第二，任何能力都要附带审计与治理机制。只有同时满足“能做”与“可控”，安全 Agent 才能持续进入真实生产流程。

9 总结与延伸

9.1 整讲框架回顾

这节课串起了三条线：评测线（什么指标值得优化）、系统线（harness 与工具如何设计）、安全线（为什么必须动态验证）。三条线最终汇聚到同一结论：**Agent 的竞争力来自系统闭环质量，而非单次文本能力。**

主题	本讲结论	可落地做法	后续挑战
评测设计	评测驱动研发方向，且有寿命	持续更新任务分布，校准真实场景	抵抗泄露与过拟合
Agent 架构	ReAct + 工具反馈是基础单元	控制流分层设计，失败轨迹必审计	成本与稳定性平衡
Harness 工程	工具接口需与模型能力共设计	ACI 思维、紧凑动作、可压缩输出	通用性与专用性折中
安全任务	必须从静态判断转向动态验证	sanitizer/debugger 进入主循环	跨项目泛化与误报控制
Big Sleep 实践	真实漏洞发现已可达成	以触发证据为目标函数做迭代	扩展到更复杂系统栈

表 5: Lecture 08 的核心结论总表

9.2 Further Reading

- Sutton 讲座中涉及的核心基线: MBPP、HumanEval、SWE-Bench、SWE-agent、AutoCodeRover。
- 安全评测与数据：公开漏洞数据库（CVE 生态）、CTF 基准、Meta 安全评测方向、DARPA 相关竞赛。
- 工程化方向：Agent Computer Interface、轨迹级失败分析、动态验证优先的 harness 设计。
- 安全研究实践：Big Sleep / Project Naptime 公开分享材料，以及与 Project Zero 协作的漏洞披露流程。

9.3 本章小结

Sutton 在收尾阶段的判断可以直接作为路线图：“*This is a wide open area. We’re just getting started.*” 对研究者和工程团队而言，下一步重点不是再做一轮表面 benchmark 刷分，而是把 Agent 放进更真实、更可审计、更可复现的安全工作流。